

ERA-Engine (Godot) 介绍

ERA-Engine Project

ERA-Engine Project 是一个最初计划用于 [ERA 游戏](#) 的现代游戏引擎的开源项目。我们希望能够创造使开发者、口上和文本作者、非专业玩家都能够轻松开发 ERA 游戏的引擎，并且支持现代程序语言和功能。

ERA-Engine (Godot)

ERA-Engine (Godot) 是以 [Godot 游戏引擎](#) 为基础的版本，也是我们当前正在主要开发的版本。

支持的语言

- [C#](#)
- [GDScript](#)

设置状态

概述

在 **FSM 模式** 下，游戏本质上是状态 **state** 间的转移，因此显而易见的，制作游戏的主要工作便是各个状态的脚本。

创造你的第一个状态

1. 创建并编辑 **main_state.gd**

2. 编写状态

- 创建一个 **main_state** 函数，作为这个状态的入口：

```
func main_state():  
    TXT("era靛紫档案") # Create a text unit  
    BOX([ # Create a series of units, pack in a box  
        BTN("开始游戏", "start_game"), # Create a button unit, and set corresponding ne  
        BTN("读取存档", "load_game")  
    ])
```

你可以使用 **BOX** 来包裹一组界面显示单元，并且用这种方式调整它们的布局。对于各个UI元素，诸如 **TXT**、**BTN**、**BOX** 等，参考API文档来查看更进一步的参数选项。

- 可以注意到的是，一个按钮 **BTN** 显然会带来状态的转换，因此在它声明时也指定了它被点击时应当转向的新状态的名字。

3. 下一个状态函数 1.使用如下代码来设置下一个状态 (**start_game** 和 **load_game**):

```
func start_game():  
    TXT("你喜欢什么样的主角? ")  
    BTN("喜爱淑女的主角", "select_char_0")  
    BTN("财大气粗的主角", "select_char_1")  
    BTN("神父气质的主角", "select_char_2")  
    BTN("流氓热血的主角", "select_char_3")
```

```
func load_game():  
    TXT("读取存档")  
    BTN("读取不了，返回", "main_state")
```

```
# Continue with select_char_0, select_char_1, select_char_2, select_char_3...
```

现在，你的 **main_state** 的按钮应该可以正常工作了。如果你希望继续完成编码工作，那么 **start_game** 部分声明了更多状态，你可以继续完善这些代码。 2. 遵循这样的工作流程，你就可以逐步构建你的第一个游戏

关于 ERA-Engine (Godot) 的进阶信息

这一章节会提供更面向专业人士或需要进一步自定义的用户的，关于 ERA-Engine (Godot) 的信息。

设计原则

有限状态机 (FSM)

数据传输对象 (DTO)

数据传输对象 (Data Transfer Object, DTO)

系统架构

进阶教程

Namespace ERAGodot.Prototype.CspScript.

Constants

Classes

[FileExtensions](#)

Enums

[ViewModels](#)